INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

CS683: Advanced Computer Architecture

Programming Assignment 2

Team QuadCore

Harsh Gupta	25M0821
Jolanda Oertli	26V0007
Amish Soni	26M0847
Vipin Chander Pal	26M0798

Department of Computer Science and Engineering
IIT Bombay

1 Task 1: Stride Pattern Prefetching

1.1 Task 1A: IP Stride Prefetcher

Objective:

The IP-stride prefetcher learns a constant cache-line stride separately for each load instruction pointer (IP). It is used as the reference prefetcher for Task 1B.

Operation:

An IP tracker stores the IP, the last cache-line address, the last observed stride, and an LRU position. On the first access from an IP, a tracker is allocated and no prefetch is generated because a stride cannot yet be calculated. On subsequent accesses, the current stride is calculated as

$$\text{stride} = \text{current cache line} - \text{previous cache line}.$$

The prefetcher issues requests only when the current stride is equal to the previous stride. For degree d, it predicts

$$A + \text{stride},\ A + 2 \cdot \text{stride}, \ldots, A + d \cdot \text{stride}.$$

Prefetches that cross the 4 KB page containing the demand access are not issued. If the L1D MSHR is less than half full, the request is filled into L1D; otherwise it is directed to L2C to reduce pressure on L1D.

Limitation:

This prefetcher recognizes only one repeated stride for an IP. It cannot learn a repeating sequence such as $+1, +2, +2, +3$, which motivates the signature-based SPP design in Task 1B.

1.2 Task 1B: Stride Pattern Prefetcher

Objective and design:

The goal is to build an L1 data-cache (L1D) prefetcher that brings data into the cache before the processor requests it. This can hide memory latency, but incorrect prefetches waste bandwidth and cache space. Our Stride Pattern Prefetcher (SPP) therefore supports two prediction modes and adjusts its aggressiveness using recent accuracy.

Per-instruction tracking:

SPP maintains an IP table with 64 entries. Each entry tracks a load instruction's IP tag, previous cache-line address, previous stride, a 2-bit uniform-pattern confidence counter, a 10-bit signature, and replacement/state information. On a new access, the stride is calculated as the current cache-line address minus the previous one. If the same stride repeats, confidence increases; if it changes, confidence is reset. Once uniform confidence reaches 3, SPP uses uniform mode and predicts future lines at that stride. For example, a stream of cache-line addresses $100, 102, 104$ suggests stride $+2$, so the next candidates are $106, 108, 110, \ldots$.

Non-uniform stride prediction:

Real access patterns do not always repeat one constant stride. A pattern such as $+1, +2, +2, +3$ cannot be represented by a single stride. When uniform confidence is insufficient, SPP uses the recent stride history to update a compact 10-bit signature:

$$\text{sig}_{new} = ((\text{sig}_{old} \ll 3)\ \mathsf{XOR}\ \text{stride}) \bmod 2^{10}.$$

The signature acts as an index into a shared 1,024-entry Signature Pattern Table (SPT). Each SPT entry stores a 7-bit predicted next stride and a 2-bit confidence counter. When the SPT confidence is high enough, its stride prediction is used to continue the pattern. Since the SPT is shared across instruction pointers, different histories can map to the same signature; this aliasing is a compactness trade-off and can sometimes cause incorrect predictions.

Candidate checks and usefulness tracking:

SPP can generate up to four future prefetch candidates per demand access. Each candidate must remain within the same 4 KB page as the demand access; candidates crossing the page boundary are dropped. A 64-entry prefetch filter prevents duplicate candidates and tracks whether an issued prefetch is later requested by a demand load. This information is used to count useful and issued prefetches and estimate accuracy. These checks limit redundant requests and provide feedback for throttling.

Accuracy-based degree control:

After every 256 accepted prefetches, SPP computes recent accuracy as useful prefetches divided by issued prefetches. The selected thresholds map accuracy to a degree from 1 to 4: degree 1 below 20%, degree 2 from 20% to below 50%, degree 3 from 50% to below 55%, and degree 4 at 55% or higher. Thus, low accuracy reduces aggressiveness, while high accuracy allows SPP to look farther ahead. The epoch of 256 requests smooths the feedback so the degree does not change based on a single prefetch outcome.

Evaluation metrics:

IPC is instructions completed per cycle (higher is better). Speedup is

$$\text{Speedup} = \frac{\text{IPC with prefetcher}}{\text{baseline IPC}}.$$

L1D Load MPKI is L1D load misses per thousand instructions (lower is better). Prefetch accuracy is the fraction of issued prefetches that are later used. Coverage is

$$\text{Coverage} = \frac{\text{Baseline L1D misses} - \text{Prefetcher L1D misses}}{\text{Baseline L1D misses}} \times 100.$$

Each simulation used 25 million warmup instructions followed by 25 million measured instructions.

Performance results:

Table 1 reports the measured values. Accuracy and coverage are not applicable to the baseline because it does not issue prefetches.

Table 1: Task 1 results for all traces.

Trace	Configuration	IPC	Speedup	L1D Load MPKI	Accuracy (%)	Coverage (%)
1	Baseline	0.232345	1.000000	10.6780	–	–
1	IP-stride	0.233452	1.004764	10.1190	52.7133	5.2354
1	SPP	0.232530	1.000796	10.5193	33.3076	1.4860
2	Baseline	0.251158	1.000000	13.5379	–	–
2	IP-stride	0.251260	1.000406	13.4378	38.1823	0.7396
2	SPP	0.251097	0.999757	13.5119	28.2994	0.1923
3	Baseline	0.382065	1.000000	3.40344	–	–
3	IP-stride	0.382862	1.002086	3.31816	15.9918	2.5057
3	SPP	0.382985	1.002408	3.19720	37.3430	6.0598
Average	Baseline	0.288523	1.000000	9.20645	–	–
Average	IP-stride	0.289191	1.002417	8.95832	35.6291	2.8269
Average	SPP	0.288871	1.000987	9.07613	32.9833	2.5794

IPC comparison:

Figure 1 compares IPC across the three traces.

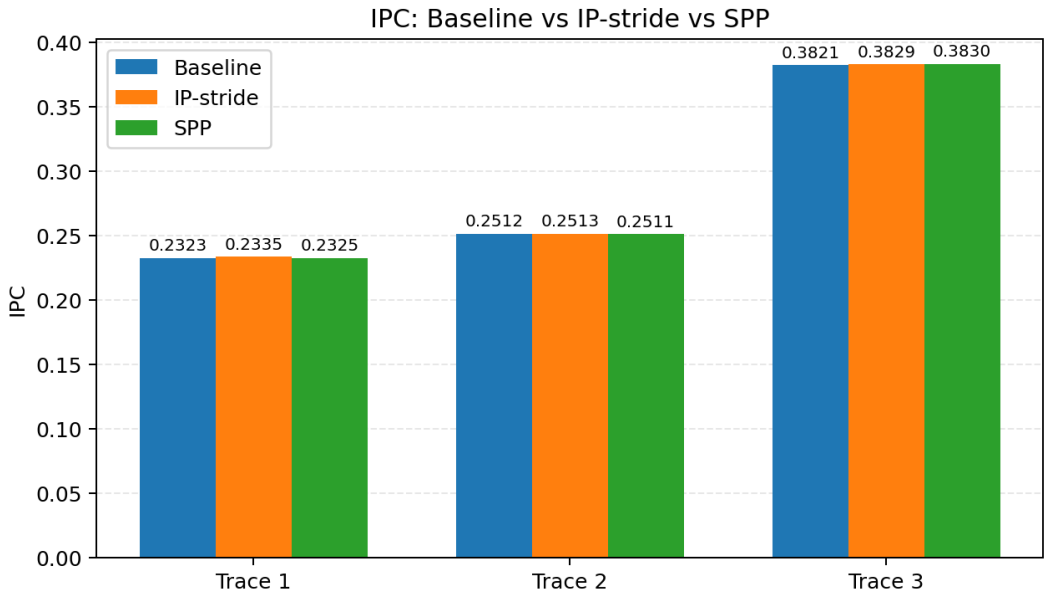


Figure 1: IPC comparison across traces.

Speedup comparison:

Figure 2 shows the speedup over the no-prefetching baseline. Values above 1 indicate an improvement over the baseline.

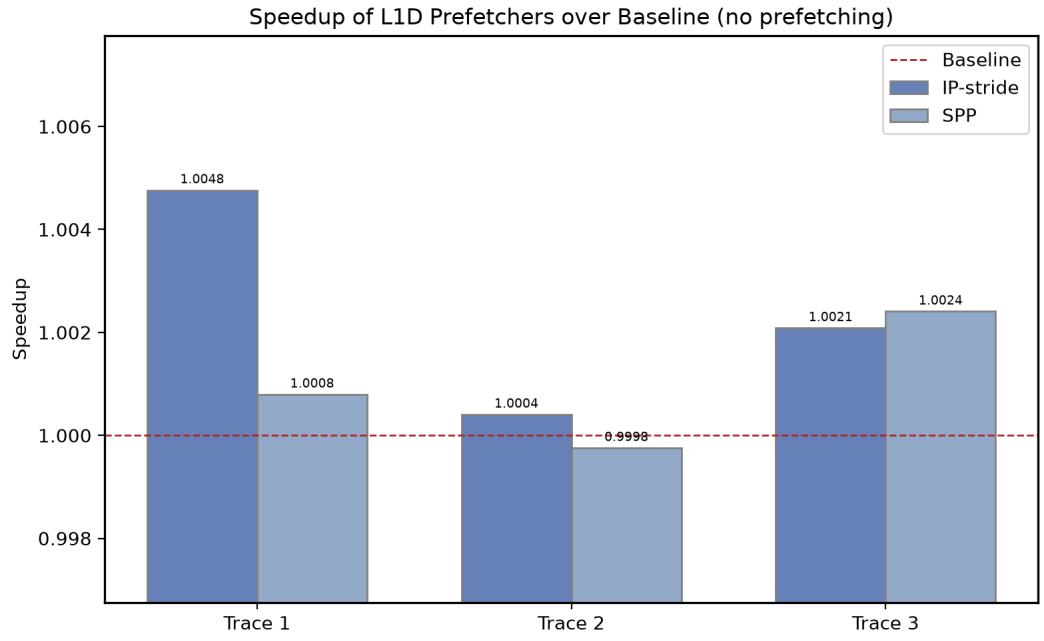


Figure 2: Speedup over the no-prefetching baseline. The dashed line marks 1.0.

L1D load MPKI comparison:

Figure 3 compares L1D load misses per thousand instructions. Lower values indicate fewer load misses.

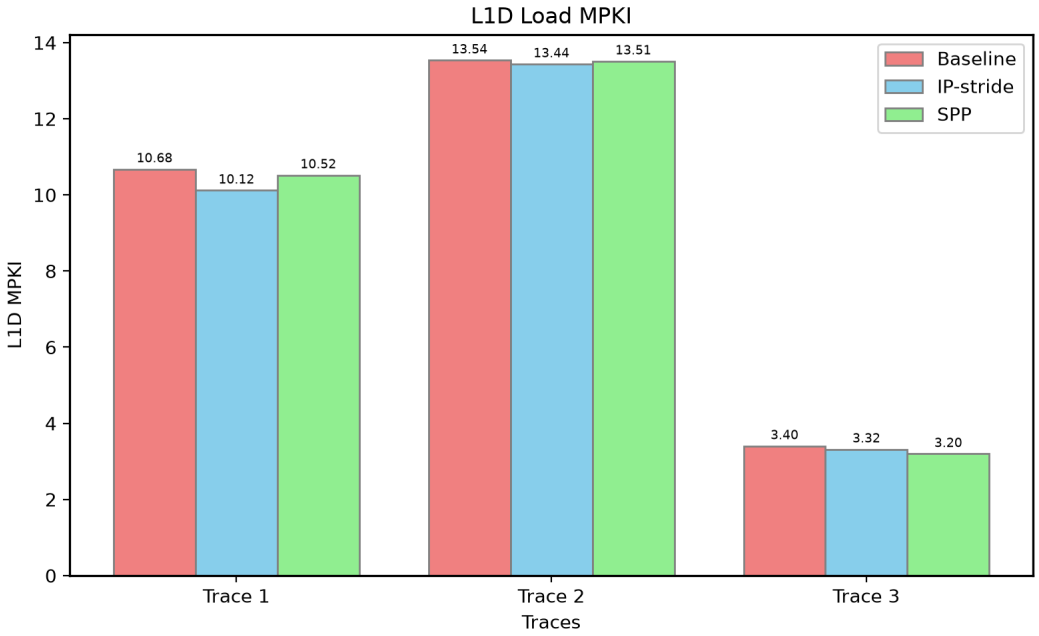


Figure 3: L1D load MPKI comparison. Lower is better.

Prefetch accuracy comparison:

Figure 4 shows the fraction of issued prefetches that are subsequently used by demand accesses.

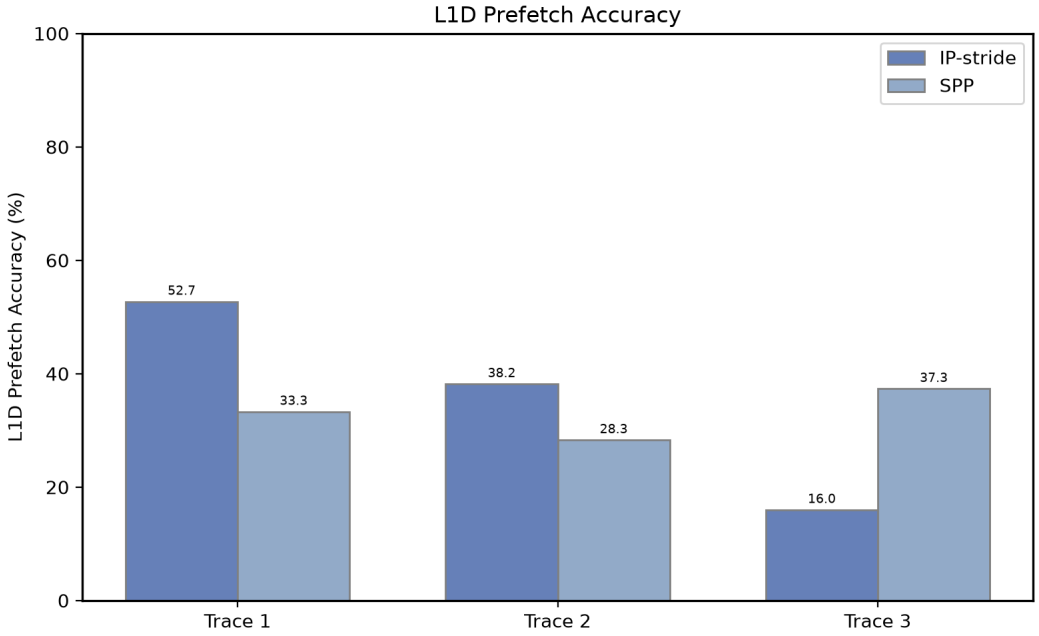


Figure 4: L1D prefetch accuracy comparison.

Prefetch coverage comparison:

Figure 5 compares the reduction in L1D load misses achieved by each prefetcher relative to the baseline.

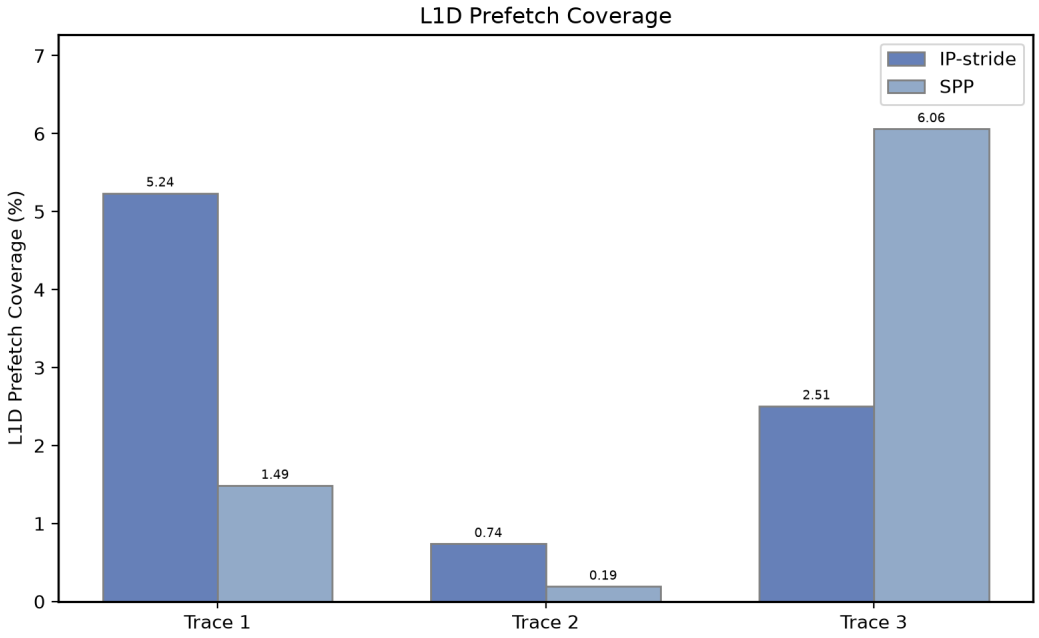


Figure 5: L1D prefetch coverage comparison.

Storage Overhead:

The following is the logical hardware-state cost, counting each field at its intended bit width and excluding C++ alignment/padding.

Table 2: Exact bit-level storage breakdown.

Structure	Field	Bits per entry	Total bits
IP table (64 entries)	valid, has_sig	$1+1$	128
	IP tag	16	1,024
	previous line	16	1,024
	signature	10	640
	previous stride	7	448
	uniform confidence	2	128
	LRU position	6	384
	IP table subtotal	**59**	**3,776**
SPT (1,024 entries)	predicted stride	7	7,168
	confidence	2	2,048
	SPT subtotal	**9**	**9,216**
Prefetch filter (64 entries)	valid, used	$1+1$	128
	tag	12	768
	epoch ID	8	512
	Filter subtotal	**22**	**1,408**
Accuracy controller	issued counter	9	9
	useful counter	9	9
	degree	3	3
	epoch ID	8	8
	Controller subtotal	–	**29**
		Total logical storage	**14,429 bits**

This equals 1,803.625 bytes, or approximately 1.76 KiB. It is a logical bit count; actual C++ allocation may be larger because of data-type alignment and padding.

Accuracy Threshold Selection:

We swept the high, mid, and low thresholds and compared mean IPC, L1D Load MPKI, prefetch accuracy, and useful/issued ratio across the three traces. The sweep indicates that $(\text{HIGH}, \text{MID}, \text{LOW}) = (55, 50, 20)$ is a good balanced setting. The tested configurations $(55, 50, 10)$, $(55, 50, 20)$, and $(55, 50, 25)$ produced the same measured results in the sweep; we select 20 for the low threshold because it gives a simple safety rule: below 20% accuracy, use the minimum degree.

High-threshold comparison:

The following graph compares the tested high-threshold configurations.

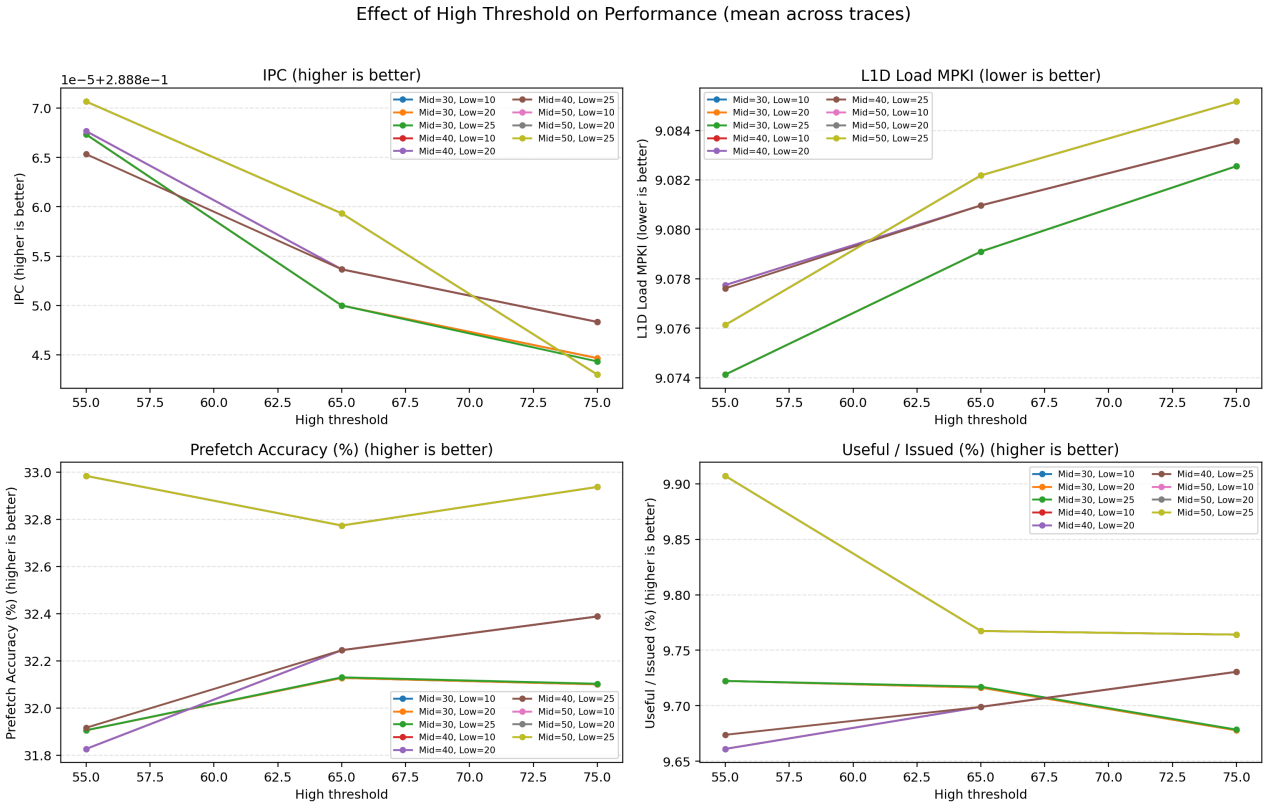


Figure 6: Threshold sweep: varying the high threshold.

Mid-threshold comparison:

The following graph compares the tested mid-threshold configurations.

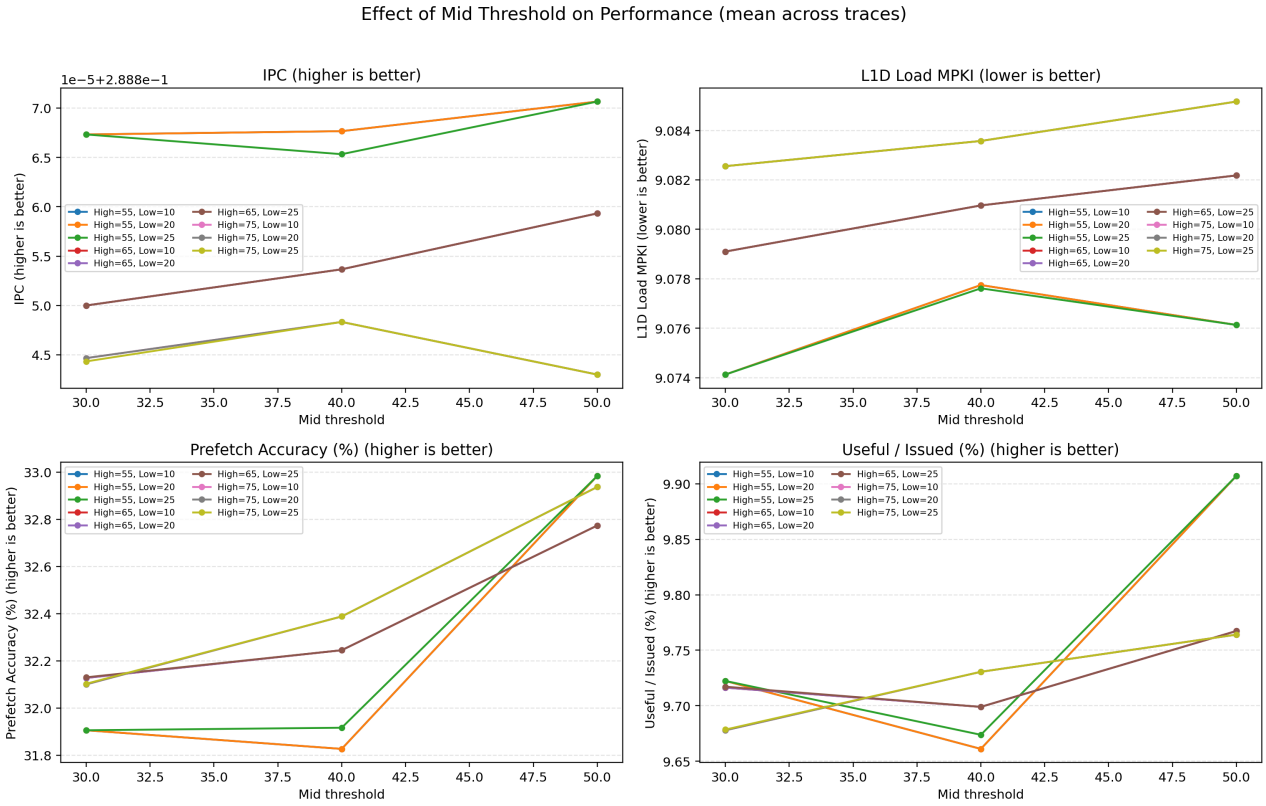


Figure 7: Threshold sweep: varying the mid threshold.

Low-threshold comparison:

The following graph compares the tested low-threshold configurations.

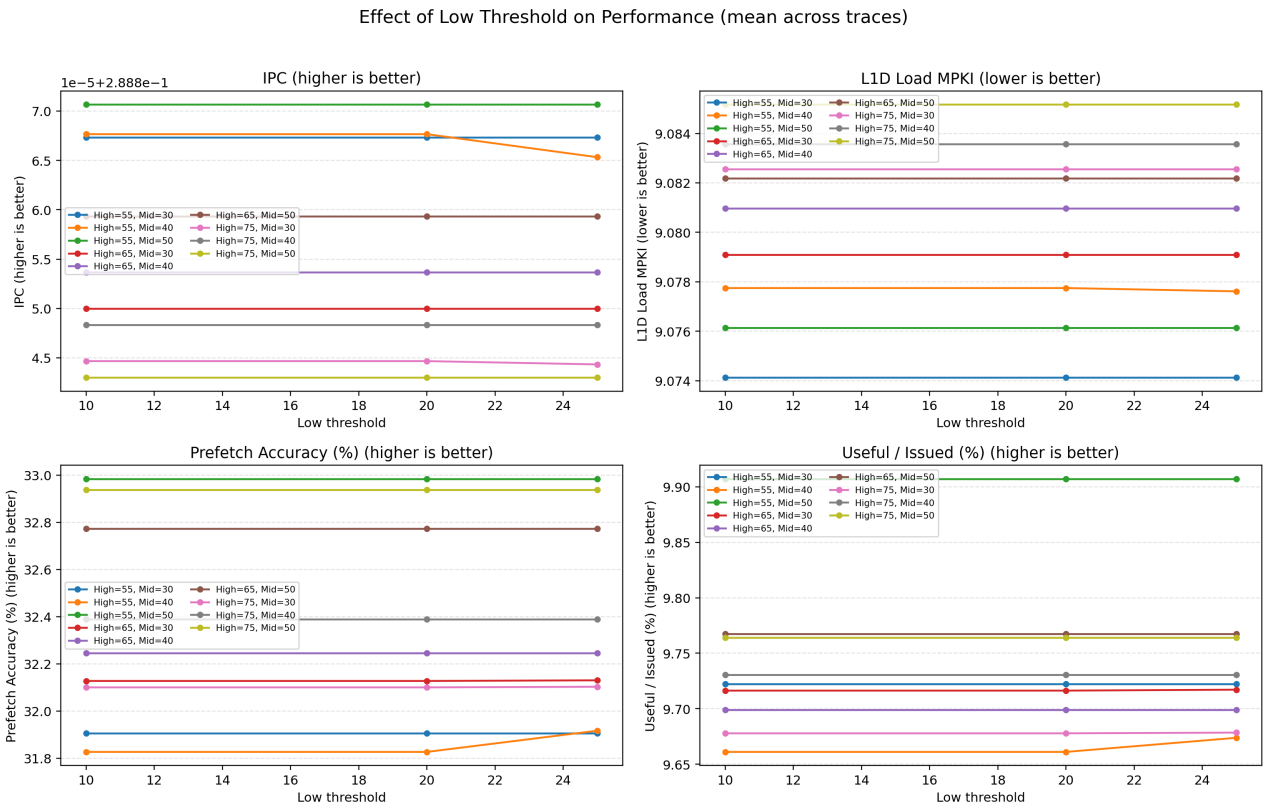


Figure 8: Threshold sweep: varying the low threshold.

Selected degree policy:

The selected policy is

$$d(a) = \begin{cases} 4, & a \geq 55\%, \\ 3, & 50\% \leq a < 55\%, \\ 2, & 20\% \leq a < 50\%, \\ 1, & a < 20\%, \end{cases}$$

where a is measured accuracy. Accuracy is updated after every 256 accepted prefetches. This epoch avoids reacting to a single request while still allowing the degree to adapt during execution.

Choice of prefetch degrees:

Degree is the number of future lines requested for one demand access. Degree 1 is conservative and limits wasted bandwidth and cache space when accuracy is poor. Degree 2 allows useful prefetching at moderate accuracy. Degrees 3 and 4 expose more future requests when recent results indicate better accuracy, potentially hiding more memory latency. Aggressive degrees can also waste bandwidth, occupy miss resources, and evict useful lines; therefore, the degree increases only when measured accuracy supports it.

Discussion and conclusion:

On average, IP-stride has the highest IPC (0.289191), lowest L1D Load MPKI (8.95832), highest accuracy (35.63%), and highest coverage (2.83%) among the two prefetchers. SPP is better on Trace 3, where it gives 37.34% accuracy and 6.06% coverage, compared with 15.99% and 2.51% for IP-stride. On Traces 1 and 2, IP-stride performs better. This suggests that SPP's ability to learn non-uniform stride sequences helps some workloads but does not guarantee an improvement on every trace. The final thresholds provide an accuracy-based way to control aggressiveness and reduce pollution.

2 Task 2: L2 Replacement Policies

2.1 Task 2A: I-RRIP

Server-style workloads have instruction footprints far larger than the L1I cache, so instruction fetches regularly miss in the L1I and are served by the L2. During an L2 instruction miss no new instructions are fetched or decoded and once the reorder buffer drains, nearly the whole miss latency is exposed. In contrast, a data miss can be partly overlapped with independent work by the out of order core. The I-RRIP developed in this task therefore deliberately trades some additional data misses for fewer instruction misses. The optimisation target is the L2 instruction load MPKI and IPC is used to verify that the trade is beneficial overall.

2.1.1 Implementation

To identify instruction type accesses, a ninth parameter `uint8_t instruction` was added to the replacement function `update_replacement_state()` which corresponds to the instruction field in the cache block/packet struct.
A line is classified as an instruction line if `instruction` is set and the access type is `LOAD` or `PREFETCH`. Page table walks for the ITLB (type `LOAD_TRANSLATION`) also carry `instruction = 1`, but they fetch page table entries, i.e. data and are treated as data. Each line holds a 1-bit instruction flag. It is set on a fill by an instruction access or by a later instruction hit to a line that was filled as data and never cleared by a data or writeback hit. The provided SRRIP implementation acts as a baseline and was modified as follows to prioritize instructions.

Insertion:

On a miss the instruction flag of the filled way is set according to the access and the RRPV is initialised to 2 for an instruction line and to 3 for a data line. A writeback that misses in the L2 is an ordinary

data fill.

Promotion:

On a hit by an instruction access the flag is set. The RRPV is then set to 0 if the line is an instruction line and to 2 if it is a data line. A writeback hit leaves the RRPV unchanged as it signals that the L1D evicted the line, not that the core re-referenced it.

Eviction and aging:

The victim search first returns an invalid way if one exists and otherwise the first data line with RRPV 3 and as last priority the first instruction line with RRPV 3. When no line in a set has an RRPV of 3, an aging round is triggered. A per-set counter advances modulo $P = 2$, incrementing data lines in every round and instruction lines only when the counter wraps to 0. If a set consists entirely of instruction lines, they are incremented every round, this speeds up search termination and still preservers their relative order.

Table 2.1.1 summarizes the difference between SRRIP and I-RRIP.

Part	**SRRIP**	**I-RRIP**
Insertion	RRPV 2	instruction at RRPV 2, data at RRPV 3
Promotion	hit → RRPV 0	instruction hit → 0, data linehit → 2, write-back hit: no change
Victim selection	first line with RRPV 3	1. invalid 2. data line before an instruction line
Aging	all lines +1	data lines +1 every round, instruction lines +1 every second round

Table 3: SRRIP versus the final I-RRIP

Storage Overhead:

Table 2.1.1 lists the hardware state for the 1024-set, 8-way L2.

Structure	**Bits**
RRPV, 2 bits/line	16 384
Instruction flag, 1 bit/line	8 192
Aging counter, $\lceil \log_2 P \rceil = 1$ bit/set	1 024
Total	25 600 (3.125 KB)
Additional storage over SRRIP	9 216 (1.125 KB)

Table 4: Storage of I-RRIP (L2: 1024 sets × 8 ways = 8192 lines)

2.1.2 Results

Simulations were performed on the single core ChampSim setup across three traces, evaluating IPC speedup and MPKI against LRU and SRRIP baselines. To isolate optimal design choices, a parameterized implementation was evaluated through an 8 combination ablation study and a 108 configuration full factorial grid search across insertion, promotion, aging and eviction parameters. The final submission only contains the best performing configuration.

Table 2.1.2 and Figure 9 report IPC, speedup and the L2C, L2 instruction and L2 data load MPKI per trace.

I-RRIP is 2.1% faster than SRRIP on average (Table 2.1.2, Figure 9) and 4.8% on Trace2, the trace with the largest instruction footprint. L2 instruction MPKI drops by 41.0% (from 9.85 to 5.81), yielding a 12.2% net drop in load MPKI. The L2C total MPKI rises from 20.60 to 23.21, but IPC improves because the avoided instruction misses are more expensive than the added data misses.

Trace	Policy	IPC	Speedup vs. LRU	Speedup vs. SRRIP	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace1	LRU	0.2323	1.0000	1.0005	17.696	10.338	7.358
	SRRIP	0.2322	0.9995	1.0000	17.639	10.562	7.077
	I-RRIP	0.2337	1.0060	1.0065	18.645	8.903	9.743
Trace2	LRU	0.2512	1.0000	0.9974	27.357	18.853	8.504
	SRRIP	0.2518	1.0026	1.0000	25.746	17.781	7.965
	I-RRIP	0.2639	1.0506	1.0479	19.166	7.878	11.289
Trace3	LRU	0.3821	1.0000	1.0009	3.954	1.124	2.830
	SRRIP	0.3817	0.9991	1.0000	4.039	1.197	2.842
	I-RRIP	0.3851	1.0080	1.0088	3.842	0.652	3.190
Mean	LRU	0.2815	1.0000	0.9996	16.336	10.105	6.231
	SRRIP	0.2816	1.0004	1.0000	15.808	9.847	5.961
	I-RRIP	0.2875	1.0213	1.0209	13.885	5.811	8.074

Table 5: IPC, speedup and L2 load MPKI of LRU, SRRIP and I-RRIP; Mean: geometric mean for IPC and speedup, arithmetic mean for MPKI

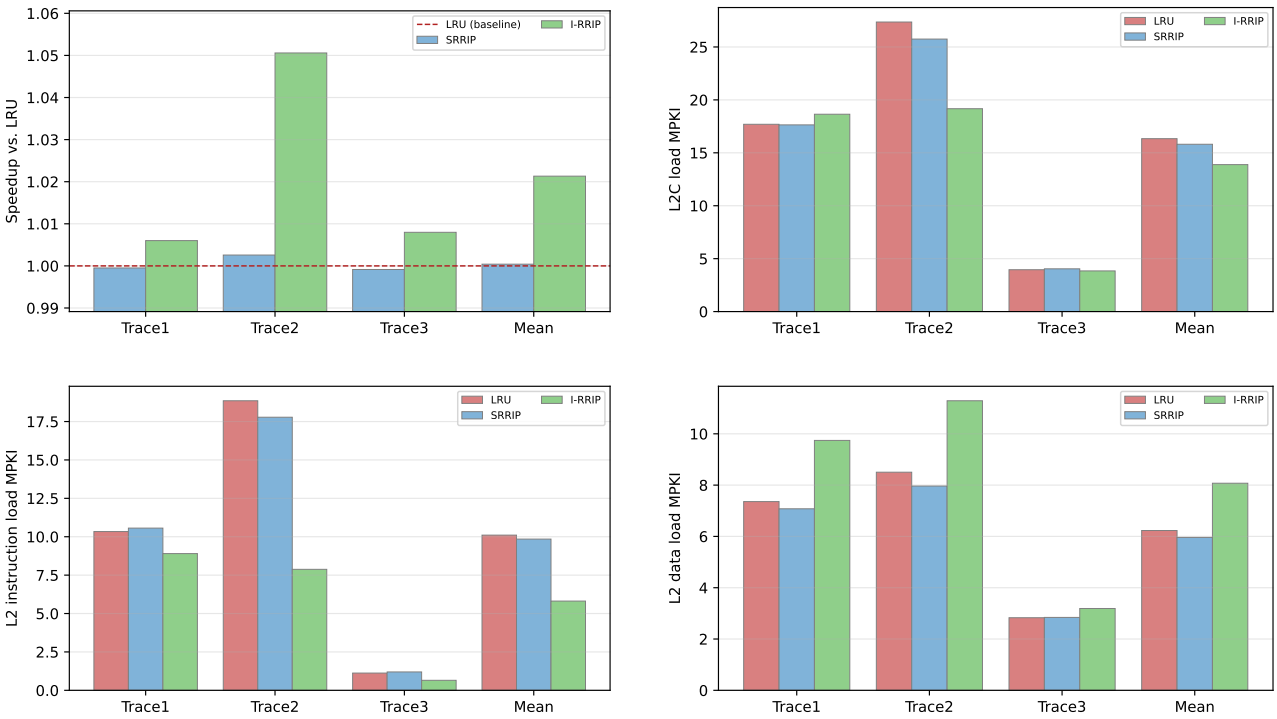


Figure 9: Task 2A: speedup over LRU (top left), L2C load MPKI (top right), L2 instruction load MPKI (bottom left) and L2 data load MPKI (bottom right)

All 108 grid configurations are faster than SRRIP (geometric-mean speedup between 1.0035 and 1.0209), and 40 of them are faster on all three traces. Instruction awareness is therefore beneficial independently of the exact parameters; the parameters determine the size of the benefit.

Insertion:

Table 6 shows the mean speedup of each insertion pair over the 18 grid configurations that share it. For every instruction insertion value, inserting data at RRPV 3 instead of 2 reduces the instruction MPKI by 0.99 to 1.22 and increases the data MPKI by 1.11 to 1.28. The L2C load MPKI changes only slightly (+0.06 to +0.16), meaning the number of misses stays about the same while their composition shifts from instruction to data misses. The speedup increases in all three cases, because an instruction miss stalls the front end while a data miss is partly overlapped by the out of order core. Distant insertion of data costs comparatively few data hits because the L2 only receives data that missed in the L1D. The L1D has already captured most short term reuse, so many data lines filled into the L2 are not referenced again at the L2. For both data insertion values, inserting instruction lines at RRPV 2 gives the lowest instruction MPKI and the lowest data MPKI. Inserting them at RRPV 1 or 0 increases both, meaning a higher insertion priority therefore does not protect code more effectively. A newly filled instruction line has not yet shown reuse, so if inserted at RRPV 0 or 1 and aged at half speed, such lines remain in the set for many aging rounds and displace instruction lines that are actually re-referenced. Thus Instruction lines receive their priority through promotion after a re-reference and not at insertion.

Promotion:

Instruction hits promote to RRPV 0 as in SRRIP, whereas data hits promote to RRPV 2. Standard SRRIP promotion keeps re-referenced data at RRPV 0 for three aging rounds, causing it to outlast and evict unreferenced instruction lines. Setting data hit RRPV to 2 allows reused data to age in just one round, still remaining prioritized over newly inserted data (RRPV 3) while yielding a 5.3% reduction in instruction MPKI. Writeback hits leave RRPV unchanged because an L1D eviction does not signal a core re-reference.

Eviction:

With SRRIP aging (period 1), instruction and data lines reach RRPV 3 in the same aging round, so once the older data candidates are evicted, instruction lines are the next victims. Aging instruction lines every second round reduces the instruction MPKI from 6.249 to 5.971 (−4.4 %) at a data MPKI increase of 0.199 and gives the lowest L2C load MPKI in this group (13.852). Period 4 reduces the instruction MPKI by a further 0.115 but increases the data MPKI by 0.130 and the L2C load MPKI by 0.014, for a speedup gain of 0.07 %. Thus period 2 was chosen, it has the lowest total number of load misses and needs 1 instead of 2 counter bits per set. Additionally a data line among the lines with RRPV 3 is preferred on eviction.

Rejected options:

Treating ITLB page walk lines as instruction lines reduced the speedup from in the initial parameter set, meaning these lines compete with code lines for priority but cannot reduce the instruction MPKI.

2.2 Task 2B: Dynamic I-RRIP

Static I-RRIP increases the L2 data MPKI on every trace. In phases in which data reuse dominates, the prioritisation of instruction lines therefore costs more misses than it saves. Dynamic I-RRIP selects between instruction prioritisation and regular SRRIP at run time.

2.2.1 Implementation

Every set is managed either by the SRRIP path or the I-RRIP path. The per line RRPV and instruction flag and the per set aging counter are maintained in both modes, so a set that switches to I-RRIP immediately knows which of its lines are instruction lines. The policy of a set is evaluated on every

Part	Value	Speedup	Instr.	Data	Load
Insertion (instr./data RRPV)	0 / 2	1.0068	7.514	6.919	14.433
	0 / 3	1.0105	6.528	8.025	14.552
	1 / 2	1.0081	7.360	6.792	14.152
	1 / 3	1.0130	6.356	7.956	14.312
	2 / 2	1.0106	7.246	6.580	13.826
	2 / 3	**1.0197**	**6.025**	7.858	13.883
Data-hit RRPV	0 (SRRIP)	1.0188	6.193	7.700	13.893
	1	1.0197	6.017	7.857	13.874
	2	**1.0205**	**5.867**	8.017	13.883
Instr. aging period	1 (SRRIP)	1.0184	6.249	7.682	13.932
	2	1.0200	5.971	7.881	**13.852**
	4	1.0207	5.856	8.011	13.866
Data-first victim	off	1.0195	6.085	7.808	13.893
	on	**1.0198**	**5.966**	7.908	**13.874**

Table 6: Mean speedup over SRRIP and mean L2 instruction, data and L2C load MPKI per parameter value. Final choice in bold.

access, for victim selection as well as for insertion and promotion. The 1024 sets are divided into N regions of $C = 1024/N$ consecutive sets. In region r, the set at offset $r \bmod C$ is an SRRIP leader and the set at offset $C - 1 - (r \bmod C)$ is an I-RRIP leader. All other sets are followers. For the final choice of $N = 16$ ($C = 64$) the SRRIP leaders are sets $65r$ and the I-RRIP leaders are sets $63r + 63$, $r = 0, \ldots, 15$. The leaders are distributed over the whole index range and use every offset once, so they do not share address bits that could make them unrepresentative. In hardware, the classification is a comparison of the upper four set-index bits with the lower six bits (SRRIP leader) or with their complement (I-RRIP leader) and requires no storage.

Decision logic, Mode 0:

Only the SRRIP leaders exist. Four counters accumulate over all demand accesses (writebacks excluded) to these sets, the instruction accesses a_I, instruction misses m_I, total accesses a and total misses m. When a reaches the epoch length E, the followers are set to I-RRIP if

$$(a_I + m_I) \cdot 100 \;>\; T \cdot (a + m), \tag{1}$$

where T is the threshold in percent (to avoid a division). All four counters are then halved, so that recent behaviour dominates and the counters are bounded by E.

Decision logic, Mode 1:

Both leader groups exist. A saturating counter PSEL starts at its midpoint. A demand miss in an SRRIP leader increments a saturating counter PSEL by c, a miss in an I-RRIP leader decrements it, with $c = W$ for instruction and $c = 1$ for data misses. Followers use I-RRIP while PSEL is above the midpoint and SRRIP otherwise. Mode 0 measures how instruction intensive the phase is and mode 1 measures which policy currently causes fewer (weighted) misses.
The final configuration is mode 1 with $W = 1$, an 8-bit PSEL and $N = 16$ leaders per group, which adds 8 bits to the Task 2A state.

2.2.2 Results

The decision parameters were swept with the same simulation setup as Task 2A. Mode 0 was tested on five thresholds $T \in \{0.10, 0.25, 0.40, 0.55, 0.70\}$ and three epoch lengths $E \in \{128, 512, 2048\}$ (32 leader sets) and mode 1 with $W \in \{1, 2, 4, 8\}$, PSEL widths of 6, 8, 10 and 12 bits and $N \in \{16, 32, 64\}$ leader sets per group (48 configurations).

Mode 0:

Table 2.2.2 and Figure 10 show the results.

T	E	**Speedup vs. SRRIP**	**Trace1**	**Trace2**	**Trace3**	**L2C load MPKI**	**Instr. MPKI**	**Data MPKI**
0.10	128	1.0160	1.0060	1.0413	1.0013	13.757	6.057	7.700
0.25	128	1.0144	1.0037	1.0388	1.0010	13.776	6.243	7.533
0.40	128	1.0126	1.0015	1.0365	1.0001	13.599	6.495	7.105
0.55	128	1.0108	1.0008	1.0318	1.0002	13.639	6.945	6.694
0.70	128	1.0088	1.0009	1.0254	1.0002	13.919	7.478	6.441
0.10	512	1.0166	1.0063	1.0419	1.0021	13.741	6.014	7.727
0.25	512	1.0159	1.0061	1.0407	1.0013	13.781	6.135	7.646
0.40	512	1.0140	1.0013	1.0398	1.0012	13.576	6.318	7.258
0.55	512	1.0117	1.0009	1.0344	1.0003	13.525	6.776	6.749
0.70	512	1.0087	1.0009	1.0253	1.0003	13.930	7.487	6.443
0.10	2048	1.0187	1.0061	1.0422	1.0082	13.721	5.958	7.763
0.25	2048	1.0157	1.0055	1.0410	1.0011	13.760	6.099	7.662
0.40	2048	1.0145	1.0020	1.0408	1.0011	13.648	6.221	7.427
0.55	2048	1.0121	1.0009	1.0356	1.0003	13.615	6.716	6.899
0.70	2048	1.0087	1.0009	1.0253	1.0003	13.929	7.488	6.441

Table 7: Mode 0: sensitivity to threshold T and epoch length E (32 leader sets)

A lower threshold keeps the followers in I-RRIP mode more often, the instruction MPKI decreases and the data MPKI increases. The speedup decreases monotonically with the threshold for every epoch length. The L2C load MPKI is lowest at intermediate thresholds (0.40 to 0.55, minimum 13.525 at $T = 0.55$, $E = 512$), where the losses on instruction and data lines are balanced. Longer epochs give slightly higher speedups, as decisions based on more accesses switch less often due to short term fluctuations.

Mode 1:

Table 2.2.2 lists the mean effect of each parameter over the other two. With $W = 1$, PSEL compares

Parameter	**Value**	**#**	**Speedup vs. SRRIP**	**L2C load MPKI**	**Instr. MPKI**	**Data MPKI**
W	1	12	1.0136	13.709	6.928	6.781
	2	12	1.0144	13.682	6.729	6.953
	4	12	1.0159	13.854	6.405	7.449
	8	12	1.0165	13.815	6.346	7.469
PSEL bits	6	12	1.0119	13.775	6.843	6.933
	8	12	1.0151	13.751	6.616	7.135
	10	12	1.0165	13.770	6.489	7.281
	12	12	1.0169	13.763	6.459	7.304
N	16	16	1.0174	13.667	6.322	7.344
	32	16	1.0132	13.887	6.879	7.009
	64	16	1.0147	13.740	6.604	7.136

Table 8: Mode 1: mean over all configurations with the given parameter value

plain miss counts, so the selection minimises the number of misses, higher weights favour instruction lines and increase speedup and data MPKI. A 6-bit PSEL saturates quickly and switches on short-term fluctuations, from 8 bits on, the results change little. Fewer leader sets leave more sets to the selected policy, more leader sets give a less noisy estimate, 16 per group (3.1 % of all sets) gives the best mean speedup and MPKI. That 32 leaders perform worse than both 16 and 64 is not explained by the mechanism and is most likely caused by the particular leader indices in combination with the three traces.

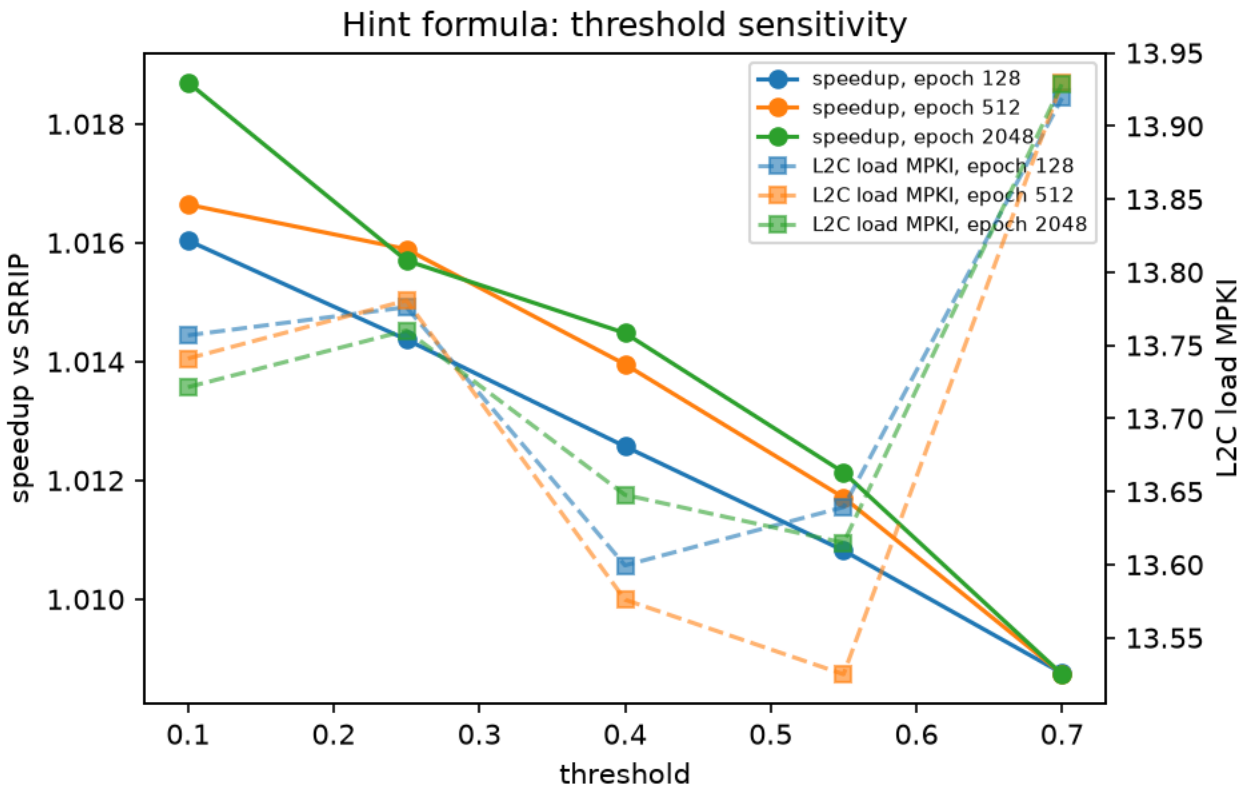


Figure 10: Mode 0: speedup and L2C load MPKI over the threshold for each epoch length.

Selection

The hint formula measures how instruction intensive the workload is and set dueling compares both policies directly on their misses. As it reaches a lower L2C load MPKI (13.479 versus 13.525 for the best threshold configuration), it was the chosen chosen approach. Within mode 1, $W = 1$, 8-bit PSEL and 16 leaders give the lowest L2C load MPKI of all 63 configurations.

Final Results:

Trace	Policy	IPC	Speedup vs. SRRIP	Speedup vs. static I-RRIP	L2C load MPKI	L2 instr. MPKI	L2 data MPKI	L2C total MPKI
Trace1	SRRIP	0.2322	1.0000	0.9935	17.639	10.562	7.077	23.851
	Static I-RRIP	0.2337	1.0065	1.0000	18.645	8.903	9.743	30.401
	Dynamic I-RRIP	0.2325	1.0010	0.9945	17.486	9.941	7.545	24.547
Trace2	SRRIP	0.2518	1.0000	0.9543	25.746	17.781	7.965	31.609
	Static I-RRIP	0.2639	1.0479	1.0000	19.166	7.878	11.289	31.059
	Dynamic I-RRIP	0.2632	1.0454	0.9976	19.116	8.382	10.735	30.183
Trace3	SRRIP	0.3817	1.0000	0.9912	4.039	1.197	2.842	6.347
	Static I-RRIP	0.3851	1.0088	1.0000	3.842	0.652	3.190	8.166
	Dynamic I-RRIP	0.3850	1.0086	0.9998	3.834	0.682	3.151	8.009
Mean	SRRIP	0.2816	1.0000	0.9795	15.808	9.847	5.961	20.602
	Static I-RRIP	0.2875	1.0209	1.0000	13.885	5.811	8.074	23.209
	Dynamic I-RRIP	0.2867	1.0181	0.9973	13.479	6.335	7.144	20.913

Table 9: IPC, speedup and L2 MPKI of SRRIP, static I-RRIP and the final Dynamic I-RRIP (set dueling, weight 1, 8-bit PSEL, 16 leader sets per group). Mean: geometric mean for IPC and speedup, arithmetic mean for MPKI.

Compared with static I-RRIP (Table 2.2.2, Figure 11), Dynamic I-RRIP reduces the mean L2C load MPKI by 2.9 % (13.885 to 13.479) and the L2C total MPKI by 9.9 % (23.21 to 20.91). The data MPKI decreases by 11.5 % and the instruction MPKI increases by 9.0 %. The speedup over SRRIP is 1.0181, 0.27 % below static I-RRIP. No dynamic configuration exceeds static I-RRIP in IPC (best 1.0198 versus 1.0209). The leader sets are bound to a fixed policy and set dueling judges the policies only by their current misses: a switch to SRRIP during a data dominated phase reduces the misses of that phase but evicts instruction lines that are reused later. Dynamic I-RRIP is therefore preferable when the number

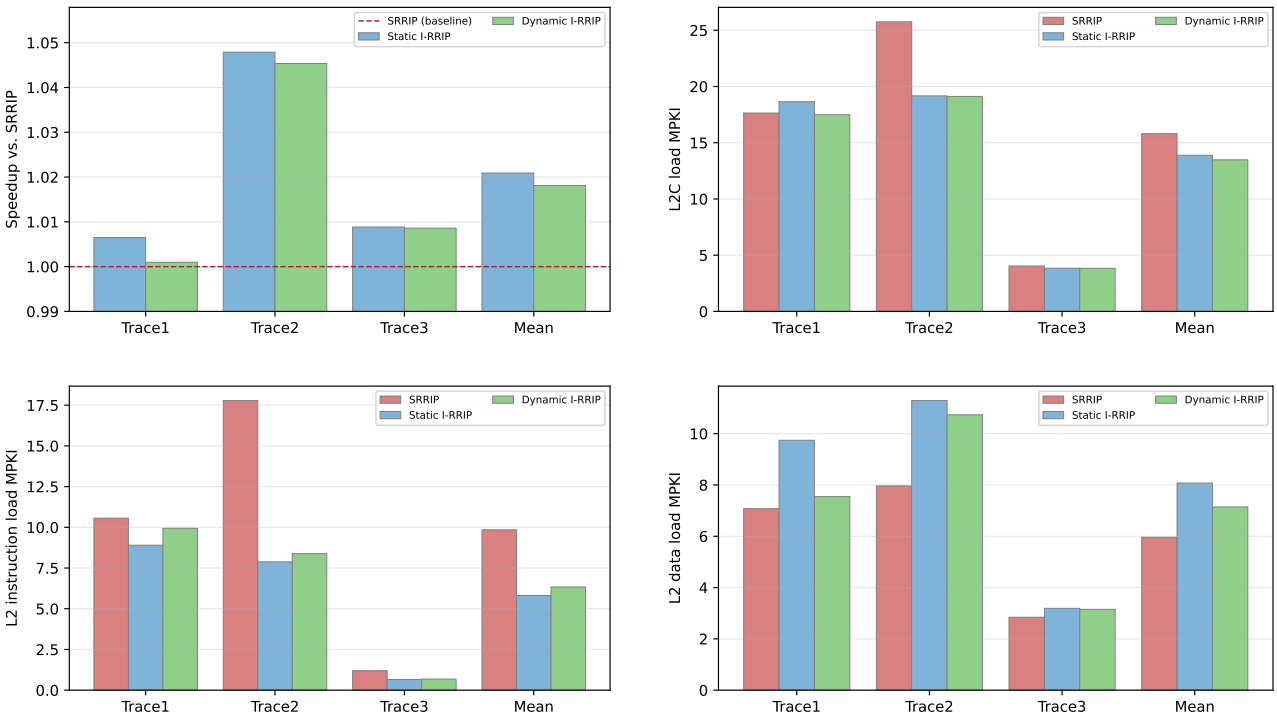


Figure 11: Speedup over SRRIP (top left), L2C load MPKI (top right), L2 instruction load MPKI (bottom left) and L2 data load MPKI (bottom right)

of L2 misses is the objective, static I-RRIP when IPC on instruction dominated workloads is.

3 Task 3: Incorporating SPP with Dynamic I-RRIP

Objective:

Task 3 combines the Stride Pattern Prefetcher (SPP) from Task 1 with the Dynamic Instruction-Conscious RRIP (Dynamic I-RRIP) replacement policy from Task 2B. SPP predicts future data accesses and brings likely-needed lines into the cache early. Dynamic I-RRIP chooses how cache lines are retained, switching between instruction-aware replacement and regular RRIP according to its selection mechanism. The goal is to make the replacement policy aware of prefetched lines so that prefetching and cache management work together.

Design and implementation:

The combined design keeps SPP responsible for predicting future data lines and uses Dynamic I-RRIP to decide which lines should remain in L2. The key change is to make the L2 replacement policy recognize whether an incoming packet is a prefetch.

- **Identify prefetched lines:** The replacement policy checks the packet type to determine whether a line was brought in by SPP. This distinguishes a predicted line from one requested directly by the processor.
- **Avoid giving prefetches excessive priority:** A prefetched line may never be used. If it is inserted with the same high priority as a repeatedly used demand line, it can evict useful instructions or data. Therefore, prefetches should be inserted with a more cautious replacement priority, allowing them to remain in the cache when useful without automatically displacing more valuable lines.
- **Keep instruction awareness:** Dynamic I-RRIP continues to select between regular RRIP and instruction-conscious replacement. The prefetch-aware handling is applied alongside this decision, rather than replacing the dynamic policy.

- **Learn from actual use:** SPP tracks whether its issued prefetches are later used and adjusts its degree based on accuracy. This limits aggressive prefetching when predictions are often incorrect.

This design aims to balance three competing needs: retaining useful instructions, making prefetched data available before demand accesses, and avoiding cache pollution from unused prefetches.

Experimental setup and metrics:

The evaluation uses the three provided traces in ChampSim. Each run uses 25 million warm-up instructions followed by 25 million measured instructions.

IPC measures instructions completed per cycle, so higher is better. Speedup is calculated relative to the no-prefetching baseline:

$$\text{Speedup} = \frac{\text{IPC with SPP+Dynamic I-RRIP}}{\text{Baseline IPC}}.$$

MPKI is the number of misses per thousand instructions; lower values are better. Prefetch accuracy is the percentage of issued prefetches later used. L1D coverage measures the reduction in L1D load misses relative to baseline.

Performance results:

Table 10 summarizes the Task 3 results. The arithmetic-average row is the mean across the three traces.

Table 10: Task 3 results across the three traces.

Trace	IPC	Speedup	Acc. (%)	Cov. (%)	L1D MPKI	L2C MPKI	L2 Instr. MPKI	L2 Data MPKI
Trace 1	0.232681	1.001446	33.091	1.363	10.533	24.787	9.940	7.331
Trace 2	0.263180	1.047866	28.006	0.117	13.522	30.325	8.389	10.699
Trace 3	0.385947	1.010161	37.923	5.695	3.210	8.197	0.690	2.833
Average	0.293936	1.019824	33.007	2.392	9.088	21.103	6.339	6.954

IPC and speedup:

Figure 12 shows that Trace 3 has the highest IPC (0.385947), followed by Trace 2 (0.263180) and Trace 1 (0.232681). The speedup is above 1 for all three traces. Trace 2 benefits the most, with a speedup of 1.047866, or about 4.79% over its baseline. The average speedup is 1.019824, approximately 1.98%. These results show a positive IPC change for each trace, although the size of the benefit varies by workload.

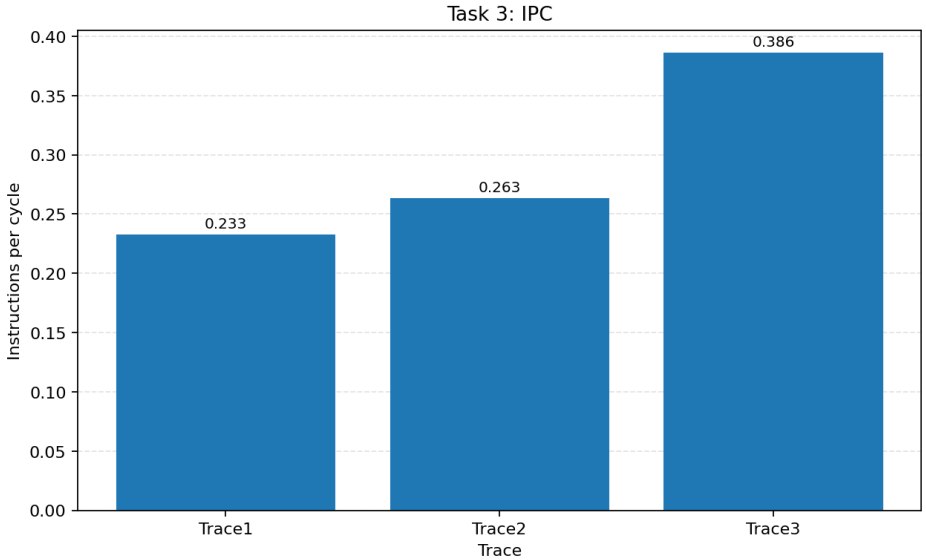


Figure 12: Task 3 IPC for each trace.

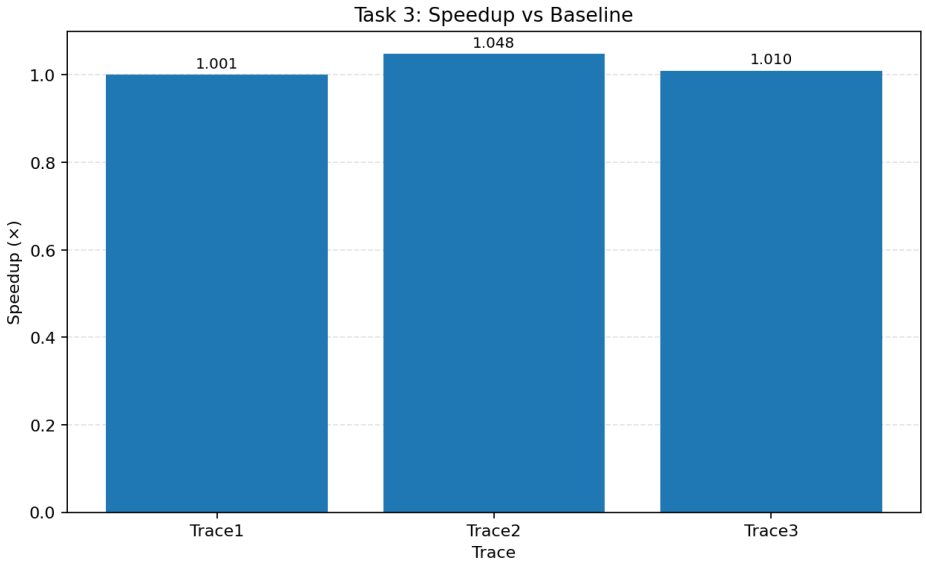


Figure 13: Task 3 speedup relative to the baseline.

L1D behavior:

The L1D load MPKI is lowest for Trace 3 (3.210) and highest for Trace 2 (13.522). Coverage is also workload-dependent: Trace 3 achieves 5.695%, Trace 1 achieves 1.363%, and Trace 2 achieves only 0.117%. This indicates that the combined design reduces L1D misses most effectively for Trace 3, while its coverage benefit is small for Trace 2.

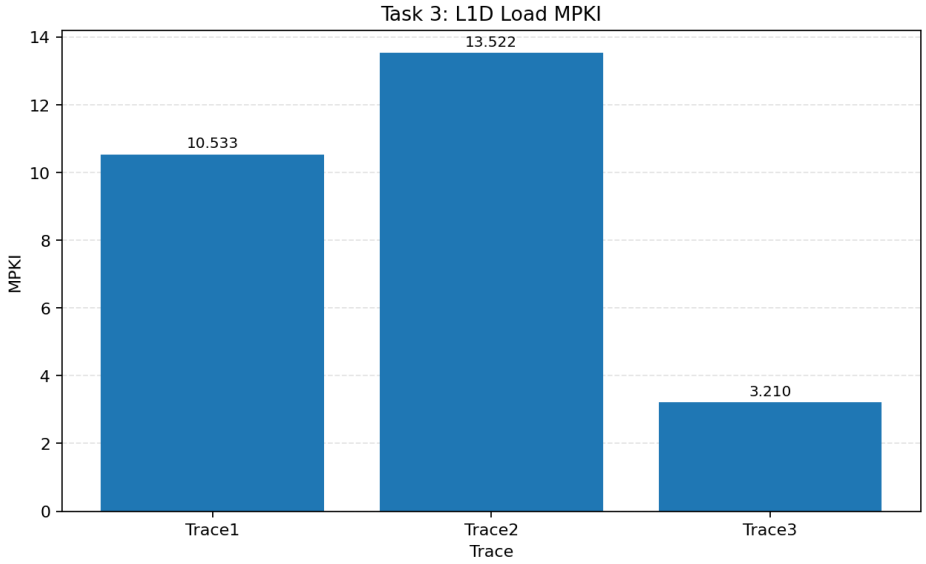


Figure 14: L1D load MPKI. Lower is better.

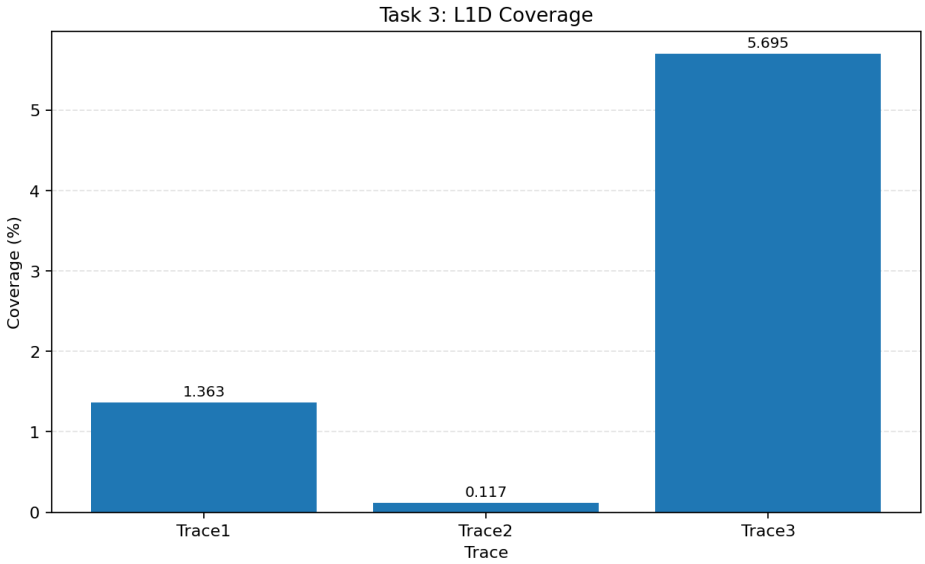


Figure 15: L1D coverage. Higher means a larger reduction in L1D load misses.

Prefetch accuracy:

Prefetch accuracy ranges from 28.006% on Trace 2 to 37.923% on Trace 3. Trace 3 has both the highest accuracy and highest coverage, suggesting that SPP's predictions are more useful for this workload. Trace 2 has lower accuracy and very low coverage, so many prefetches do not translate into a meaningful reduction in L1D load misses.

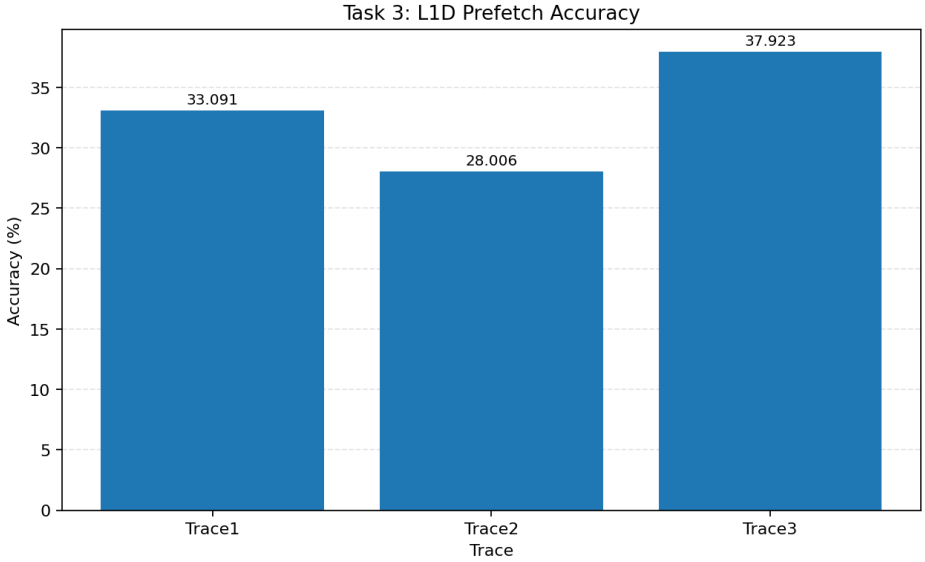


Figure 16: L1D prefetch accuracy.

L2 behavior:

Trace 2 has the highest L2C total MPKI (30.325), while Trace 3 has the lowest (8.197). The L2 instruction load MPKI is 9.940, 8.389, and 0.690 for Traces 1, 2, and 3, respectively. L2 data load MPKI is highest for Trace 2 (10.699) and lowest for Trace 3 (2.833). These values show that the cache-miss behavior differs considerably across traces; the combined policy does not produce the same miss profile for every workload.

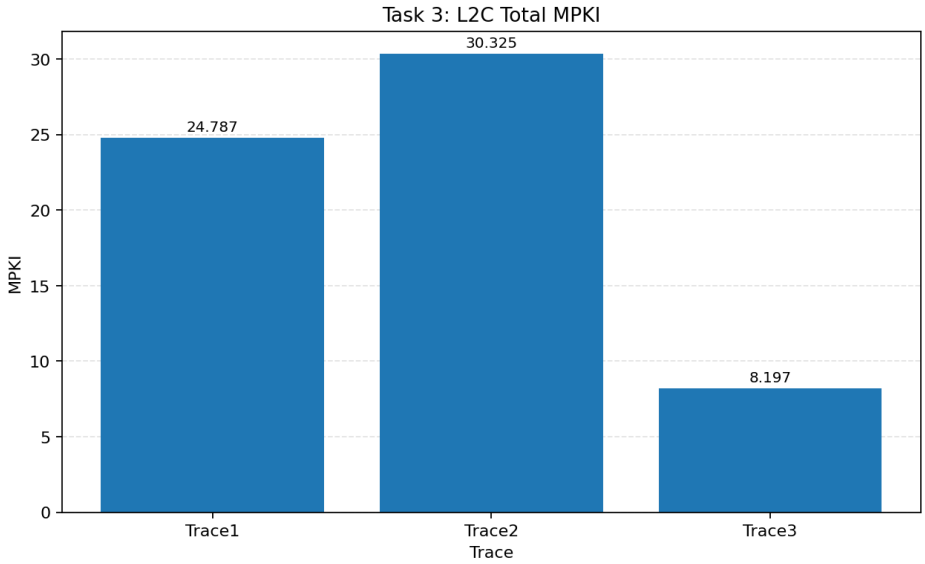


Figure 17: L2C total load MPKI. Lower is better.

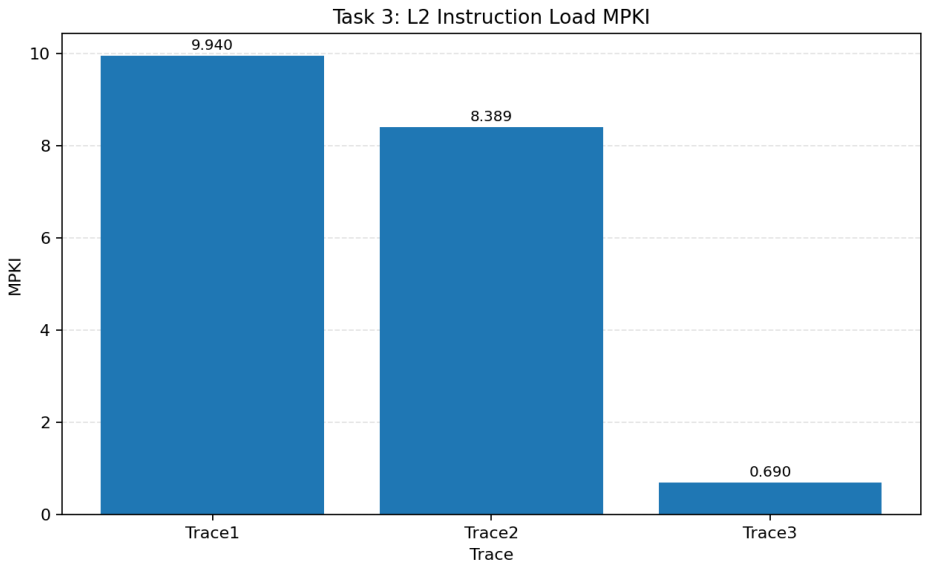


Figure 18: L2 instruction load MPKI. Lower is better.

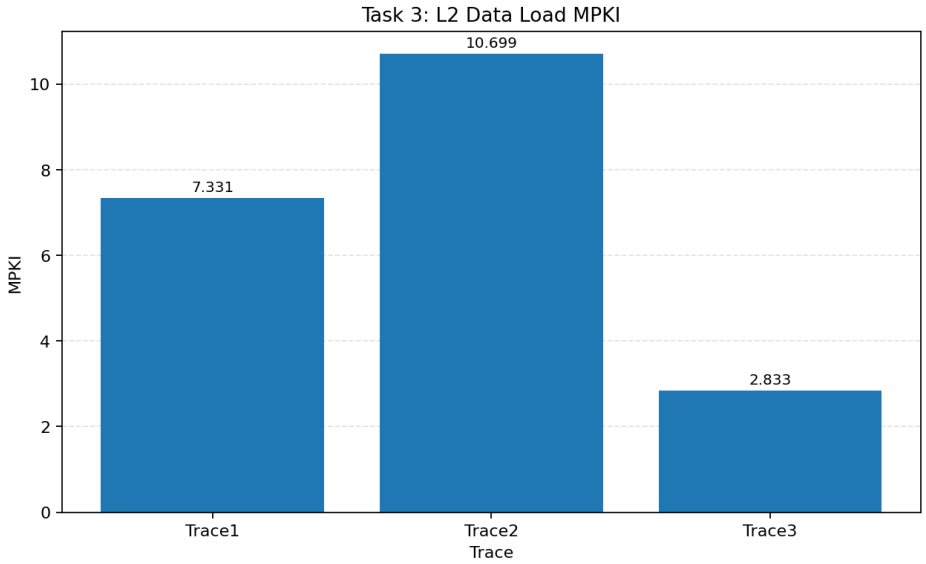


Figure 19: L2 data load MPKI. Lower is better.

Conclusion:

We combined SPP with Dynamic I-RRIP and added prefetch awareness to the L2 replacement policy. Across the three traces, the average IPC is 0.293936 and average speedup is 1.019824. Trace 2 shows the largest speedup (about 4.79%), while Trace 3 has the highest prefetch accuracy (37.923%) and coverage (5.695%). Trace 2's very low coverage (0.117%) shows that useful prefetching depends on the workload. Overall, the results show an average performance improvement over the baseline.